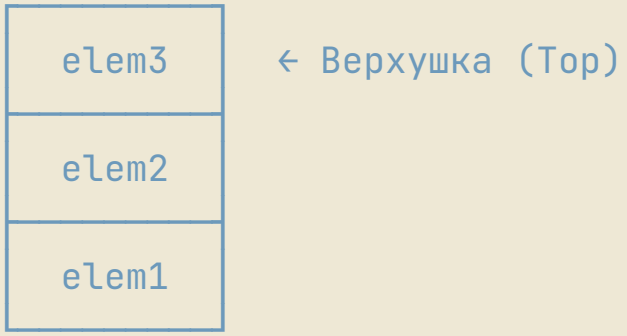
1. Структура данных Stack (Стек)

Что такое стек?

Стек — это структура данных, организованная по принципу **LIFO** (Last In, First Out — "последним пришёл, первым ушёл"). Представьте стопку тарелок: вы можете положить новую тарелку только сверху и взять тоже можно только верхнюю.



Основные операции:

- **Push(item)** — положить элемент на вершину стека
- **Pop()** — извлечь и вернуть верхний элемент
- **Peek()** — посмотреть на верхний элемент, не извлекая его
- **Count** — узнать количество элементов в стеке

Реализация своего стека на базе MyArrayList

> ⚠️ **Важно для лабораторной работы:** В этом справочнике показаны примеры с готовым `Stack<T>` из .NET для понимания принципов работы. Однако **в рамках лабораторной работы вы должны реализовать свой класс стека**, используя ваш `MyArrayList` из 5-й лабораторной работы!

Связь стека с динамическим массивом

Стек можно легко реализовать на основе динамического массива:

- **Вершина стека** — это последний добавленный элемент (элемент с индексом `Count - 1`)
- **Push** — добавление элемента в конец списка

- **Pop** — удаление последнего элемента и возврат его значения
- **Peek** — чтение последнего элемента без удаления

Визуализация:



Готовый `Stack<T>` в C# (для справки)

> 🔥 **Эти примеры показаны для понимания работы стека. В вашей лабораторной работе используйте свой класс MyStack.**

В C# стек реализован в пространстве имён `System.Collections.Generic` как обобщённый класс `Stack<T>`.

```
using System;
using System.Collections.Generic;

static void Main(string[] args)
{
    // Создание стека для хранения целых чисел
    Stack<int> numbers = new Stack<int>();

    // Добавление элементов
    numbers.Push(10);
    numbers.Push(20);
    numbers.Push(30);

    Console.WriteLine($"Элементов в стеке:
{numbers.Count}"); // 3
```

```
    // Посмотреть на верхний элемент (не удаляя)
    int top = numbers.Peek();
    Console.WriteLine($"Верхний элемент: {top}"); // 30
    Console.WriteLine($"Элементов в стеке:
{numbers.Count}"); // 3 (не изменилось)

    // Извлечь верхний элемент
    int removed = numbers.Pop();
    Console.WriteLine($"Извлечённый элемент: {removed}");
// 30
    Console.WriteLine($"Элементов в стеке:
{numbers.Count}"); // 2

    // Проверка на пустоту
    if (numbers.Count > 0)
    {
        Console.WriteLine("Стек не пуст");
    }
}
```

Пример со строками:

```
Stack<string> operators = new Stack<string>();

operators.Push("+");
operators.Push("*");
operators.Push("-");

// Извлекаем элементы в порядке LIFO
while (operators.Count > 0)
{
    Console.WriteLine(operators.Pop()); // Выведет: -, *,
+
}
```

---

2. Работа со строками и токенами

Разбиение строки на токены (Split)

Метод `Split()` разделяет строку на массив подстрок по указанному разделителю.

```
string expression = "3 + 4 * 2";
string[] tokens = expression.Split(' ');  // Разделитель - пробел

// tokens = { "3", "+", "4", "*", "2" }

foreach (string token in tokens)
{
    Console.WriteLine(token);
}
```

Разделение по нескольким символам:

```
string data = "яблоко,груша;банан апельсин";
// Разделяем по запятой, точке с запятой и пробелу
string[] fruits = data.Split(new char[] { ',', ';', ' ' });

// fruits = { "яблоко", "груша", "банан", "апельсин" }
```

> ⚠ **Важно для лабораторной работы (Часть 1 — Лексический анализатор):** `Split`, показанный ниже, годится только для строки в формате ОПН из **Части 2**, где токены по условию уже разделены пробелами. Во входном выражении для **Части 1** пробелов между токенами может не быть вообще (`"(3+4)*2"`) — `Split(' ')` в этом случае просто вернёт исходную строку одним токеном. Для Части 1 нужен посимвольный разбор (см. раздел «Посимвольный разбор строки» ниже в этом файле), а не готовый `Split`.

Удаление пустых элементов

```
string messy = "3  +  4   *  2"; // Несколько пробелов
подряд
string[] tokens = messy.Split(new char[] { ' ' },

StringSplitOptions.RemoveEmptyEntries);

// tokens = { "3", "+", "4", "*", "2" } (пустые строки
удалены)
```

Проверка строки на пустоту

```
string str1 = "";
string str2 = "   ";
string str3 = null;

// IsNullOrEmpty - проверяет на null и пустую строку
Console.WriteLine(string.IsNullOrEmpty(str1)); // true
Console.WriteLine(string.IsNullOrEmpty(str2)); // false
(есть пробелы)
Console.WriteLine(string.IsNullOrEmpty(str3)); // true

// IsNullOrWhiteSpace - проверяет также на строки из
пробелов
Console.WriteLine(string.IsNullOrWhiteSpace(str1)); //
true
Console.WriteLine(string.IsNullOrWhiteSpace(str2)); //
true (только пробелы)
Console.WriteLine(string.IsNullOrWhiteSpace(str3)); //
true
```

Удаление пробелов (Trim)

```
string dirty = "  hello world  ";
string clean = dirty.Trim(); // "hello world"

// TrimStart - удаляет пробелы только слева
string leftClean = dirty.TrimStart(); // "hello world  "
```

```
// TrimEnd - удаляет пробелы только справа
string rightClean = dirty.TrimEnd(); // "  hello world"
```

Посимвольный разбор строки (для Части 1)

Это то, что реально требуется в Части 1: пройти по строке символ за символом и самому решить, где заканчивается один токен и начинается другой — без опоры на пробелы как разделитель.

Идея алгоритма (это план, а не готовое решение — реализовать нужно самостоятельно):

```
i = 0
tokens = []
пока i < expression.Length: // длина выражения
    ch = expression[i] // ch - текущий символ

    если ch — пробел:
        i++                 // пробел просто пропускаем, он
не токен
        продолжить

    если ch — цифра или '.':
        buffer = ""
        пока i < length И (expression[i] — цифра или
'.'):
            buffer += expression[i]
            i++
        tokens.Add(buffer)   // накопленное число — один
токен
        // i уже указывает на символ ПОСЛЕ числа, дальше
i++ не нужен

    иначе если ch — один из '+','-','*','/','^','(',')':
        tokens.Add(ch.ToString())
```

```
        i++

    иначе:
        throw new FormatException($"Недопустимый символ:
'{ch}'")
```

Ключевой момент: цифры **не** добавляются в список токенов по одной — они копятся в буфер, пока идут подряд, и только когда цифры (и точка) заканчиваются, весь накопленный буфер добавляется как **один** токен-число. Так `"123"` даёт токен `"123"`, а не три токена `"1"`, `"2"`, `"3"`.

```
// Заготовка сигнатуры метода для Части 1
static List<string> Tokenize(string expression)
{
    var tokens = new List<string>();
    // ваша реализация посимвольного разбора здесь
    return tokens;
}
```

---

3. Преобразование типов данных

Строка в число

Метод Parse (небезопасный)

```
string numberStr = "123";
int number = int.Parse(numberStr); // number = 123

// Если строка некорректна - будет исключение!
string bad = "abc";
int error = int.Parse(bad); // Вылетит FormatException!
```

Метод TryParse (безопасный)

```
string input = "456";
int result;
```

```
// TryParse возвращает true, если преобразование успешно
if (int.TryParse(input, out result))
{
    Console.WriteLine($"Число: {result}"); // Число: 456
}
else
{
    Console.WriteLine("Не удалось преобразовать");
}

// Для вещественных чисел:
string floatStr = "3.14";
double value;

if (double.TryParse(floatStr, out value))
{
    Console.WriteLine($"Значение: {value}"); // Значение: 3.14
}
```

Число в строку

```
int number = 42;
string str = number.ToString(); // "42"

// Форматирование
double pi = 3.14159;
string formatted = pi.ToString("F2"); // "3.14" (2 знака после запятой)
```

---

4. Проверка типа токена

Является ли строка числом?

```
static bool IsNumber(string token)
{
    double temp;
    return double.TryParse(token, out temp);
}

// Использование
Console.WriteLine(IsNumber("123"));   // true
Console.WriteLine(IsNumber("3.14"));  // true
Console.WriteLine(IsNumber("+"));     // false
```

Является ли строка оператором?

```
static bool IsOperator(string token)
{
    return token == "+" || token == "-" || token == "*"
|| token == "/" || token == "^";
}

// Или через массив
static bool IsOperator(string token)
{
    string[] operators = { "+", "-", "*", "/", "^" };
    return Array.Exists(operators, op => op == token);
}
```

Является ли строка скобкой?

```
static bool IsOpenBracket(string token)
{
    return token == "(";
}

static bool IsCloseBracket(string token)
{
    return token == ")";
}
```

5. Приоритет операций

Определение приоритета

Вам нужно придумать способ сопоставить оператору его вес. `*` и `/` весят больше, чем `+` и `-`. Как?....

Таблица приоритетов

Оператор	Приоритет	Ассоциативность
`(` , `)`	0 (спец.)	-
`+` , `-`	1 (низкий)	Левая
`*` , `/`	2 (средний)	Левая
`^`	3 (высокий)	Правая

---

6. Обратная Польская Нотация (ОПН)

Что такое ОПН?

Обратная Польская Нотация (Reverse Polish Notation, RPN) — способ записи математических выражений, где оператор записывается **после** операндов.

Примеры:

Инфиксная нотация	Постфиксная (ОПН)	Результат
`3 + 4`	`3 4 +`	7
`5 - 2`	`5 2 -`	3
`2 * 3`	`2 3 *`	6
`(3 + 4) * 2`	`3 4 + 2 *`	14
`3 + 4 * 2`	`3 4 2 * +`	11

1. **Не нужны скобки** — порядок вычислений определяется порядком следования операндов и операторов

2. **Простота вычисления** — одним проходом слева направо с использованием стека

---

7. Алгоритм вычисления выражения в ОПН

Принцип работы

1. Создаём стек для чисел
2. Читаем токены слева направо:
   - Если **число** → кладём в стек
   - Если **оператор** → извлекаем два числа, выполняем операцию, результат кладём обратно в стек
3. В конце в стеке остаётся один элемент — это результат

Визуальный пример

Вычислим `3 4 + 2 *`:

```
Токен: 3  | Стек: [3]
Токен: 4  | Стек: [3, 4]
Токен: +  | Извлекаем 4 и 3, считаем 3+4=7 | Стек: [7]
Токен: 2  | Стек: [7, 2]
Токен: *  | Извлекаем 2 и 7, считаем 7*2=14 | Стек: [14]

Результат: 14
```

Пример обработки одного оператора

```
// Пример: как обработать один оператор "+"
Stack<double> numbers = new Stack<double>();

// Положили два числа
numbers.Push(3);
numbers.Push(4);

// Встретили оператор "+"
// ВАЖНО: порядок извлечения!
```

```
double operand2 = numbers.Pop(); // 4 (правый операнд)
double operand1 = numbers.Pop(); // 3 (левый операнд)

double result = operand1 + operand2; // 3 + 4 = 7
numbers.Push(result); // Результат обратно в стек

Console.WriteLine(numbers.Peek()); // 7
```

Обратите внимание: для операций, где порядок важен (вычитание, деление), нужно правильно извлекать операнды. Первым из стека выйдет **правый** операнд, вторым — **левый**.

---

8. Алгоритм сортировочной станции Дейкстры

Принцип работы

Алгоритм преобразует инфиксную нотацию в постфиксную (ОПН).

Основные правила:

1. Числа сразу идут в выходную строку
2. Операторы идут в стек, но с учётом приоритета
3. Открывающая скобка `(` — всегда в стек
4. Закрывающая скобка `)` — выталкивает операторы до `(`

Пошаговый пример

Преобразуем `3 + 4 * 2` в ОПН:

Шаг	Токен	Выход	Стек операторов	Действие
1	3	`3`	[]	Число → в выход
2	+	`3`	[+]	Оператор → в стек
3	4	`3 4`	[+]	Число → в выход
4	*	`3 4`	[+, *]	* приоритетнее +, кладём в стек
5	2	`3 4 2`	[+, *]	Число → в выход

Шаг	Токен	Выход	Стек операторов	Действие
6	(конец)	`3 4 2 * +`	[]	Выталкиваем все операторы

Результат: `3 4 2 * +`

Пример обработки скобок

Преобразуем `( 3 + 4 ) * 2`:

Шаг	Токен	Выход	Стек операторов	Действие
1	(	``	[(]	Открывающая скобка → в стек
2	3	`3`	[(]	Число → в выход
3	+	`3`	[(, +]	Оператор → в стек
4	4	`3 4`	[(, +]	Число → в выход
5	)	`3 4 +`	[]	Выталкиваем до `(`, саму `(` удаляем
6	*	`3 4 +`	[*]	Оператор → в стек
7	2	`3 4 + 2`	[*]	Число → в выход
8	(конец)	`3 4 + 2 *`	[]	Выталкиваем оставшиеся операторы

Результат: `3 4 + 2 *`

---

9. Обработка ошибок

Проверка на пустоту перед Pop/Peek

```
Stack<int> stack = new Stack<int>();

// NOT OKAY - может вылететь исключение. Почему? Стек пустой.
```

```
int value = stack.Pop(); // InvalidOperationException!

// OKAY - проверяем перед извлечением
if (stack.Count > 0)
{
    int value = stack.Pop();
    return value;
}
else
{
    // Сделать что то когда сломалось (Исключение).
    throw new InvalidOperationException("Опишите что
случилось");
}
```

Проверка достаточности операндов

```
// Перед применением оператора нужно убедиться,
// что в стеке есть минимум 2 числа
Stack<double> numbers = new Stack<double>();
numbers.Push(5);
// Попытка применить оператор при одном операнде -
ошибка!

if (numbers.Count < 2)
{
    // Сделать что то когда сломалось (Исключение).
    throw new InvalidOperationException("Некорректное
выражение: недостаточно операндов для операции.");
}
else
{
    double b = numbers.Pop();
    double a = numbers.Pop();
    double result = a + b;
    numbers.Push(result);
}
```

Проверка деления на ноль

```
double a = 10;
double b = 0;

if (b == 0)
{
    throw new DivideByZeroException("Деление на ноль.");
}
else
{
    double result = a / b;
    return result;
}
```

---

10. Полезные методы для работы со строками

Конкатенация строк

```
// Простое сложение (неэффективно для большого количества операций)
string result = "Hello" + " " + "World"; // "Hello World"

// StringBuilder - эффективный способ для множественных операций
StringBuilder sb = new StringBuilder();
sb.Append("3");
sb.Append(" ");
sb.Append("4");
sb.Append(" ");
sb.Append("+");
string rpn = sb.ToString(); // "3 4 +"

// String.Join - объединение массива строк
string[] tokens = { "3", "4", "+" };
string joined = string.Join(" ", tokens); // "3 4 +"
```

Поиск подстроки

```
string text = "3 + 4 * 2";

// Contains - проверка наличия подстроки
bool hasPlus = text.Contains("+"); // true

// IndexOf - получение позиции первого вхождения
int position = text.IndexOf("*"); // 6
if (position == -1)
{
    Console.WriteLine("Символ не найден");
}

// StartsWith / EndsWith
bool startsWithThree = text.StartsWith("3"); // true
bool endsWithTwo = text.EndsWith("2"); // true
```

Замена подстрок

```
string expression = "3+4*2";

// Replace - замена всех вхождений
string spaced = expression.Replace("+", " + ")
                          .Replace("*", " * ");
// Результат: "3 + 4 * 2"
```

Полезные ссылки:

1. `Stack<T>` – https://learn.microsoft.com/ru-ru/dotnet/api/system.collections.generic.stack-1
2. `String.Split` – https://learn.microsoft.com/ru-ru/dotnet/api/system.string.split
3. `Double.TryParse` – https://learn.microsoft.com/ru-ru/dotnet/api/system.double.tryparse

4. `Math` – https://learn.microsoft.com/ru-ru/dotnet/api/system.math
5. `StringBuilder` – https://learn.microsoft.com/ru-ru/dotnet/api/system.text.stringbuilder
6. `List<T>` – https://learn.microsoft.com/ru-ru/dotnet/api/system.collections.generic.list-1